Question 1: Programming: Write a Program - Stacks

Given a string s containing the characters '(' and ')' determine if the input string is valid.

An input string is valid if:

- Open parentheses must be closed in the correct order.
- Every close parenthesis has a corresponding open parenthesis that comes before it.

Example 1:

Input: s = "()"
Output: True

Example 2:

Input: s = "())("
Output: False

Example 3:

Input: s = ")()()("
Output: False

Example 4:

Input: s = "(())()"
Output: True

Example 5:

Input: s = “()()(“
Output: False

Question 2: Programming: Write a Program - Linked Lists

We want to write code to partition a linked list around a value x, such that all nodes less than x come before all nodes greater than or equal to x. If x is contained within the list, the values of x only need to be after the elements less than x (see below). The partition element x can appear anywhere in the "right partition"; it does not need to appear between the left and right partitions. Either return a new list or modify the existing.

Example
Input: 3 -> 5 -> 8 -> 5 -> 10 -> 2 -> 1 [partition = 5]
Output: 3 -> 1 -> 2 -> 10 -> 5 -> 5 -> 8

Complete the code:

```
class Node:
   """A simple node for a singly linked list."""
   def __init__(self, data=0, next_node=None):
       self.data = data
       self.next = next_node

class LinkedList:
   """A simple linked list class for demonstration purposes."""
   def __init__(self):
       self.head = None

   def append(self, data):
       """Appends a new node with the given data to the end of the list."""
       if not self.head:
           self.head = Node(data)
           return
       current = self.head
       while current.next:
           current = current.next
       current.next = Node(data)

   def print_list(self):
       """Prints the elements of the list."""
       nodes = []
       current = self.head
       while current:
           nodes.append(str(current.data))
           current = current.next
       print(" -> ".join(nodes))
```

```
def partition(node, x):
    """
    Partitions a linked list around a value x, such that all nodes less than x
    come before all nodes greater than or equal to x.


    Args:
  node: The head node of the linked list.
        x: The partition value.


    Returns:
        The head of the new partitioned linked list.
    """
    before_start = None
    before_end = None
    after_start = None
    after_end = None


    # Partition the list into two: one for elements <x and one for elements >= x
    while node is not None:
        next_node = node.next
        node.next = None  # Disconnect the node from the original list


        # COMPLETE THE LOOP BY INSERTING THE NODE INTO THE END OF EITHER THE ‘before’
LIST OR THE ‘after’ LIST

        node = next_node

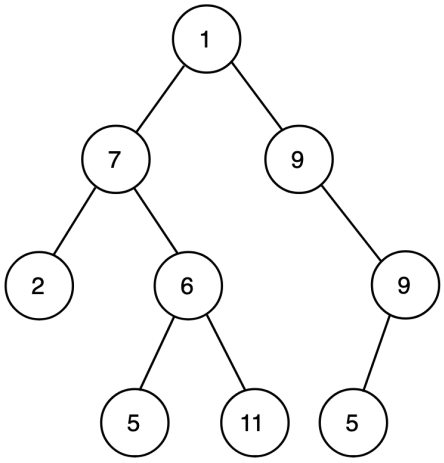
    if before_start is None:
        return after_start

    # Merge the 'before' and 'after' lists

    return before_start
```

Question 3: Code-Tracing - Binary Trees

3a) Trace the following code segment. The input is the tree in the picture below. What is the output? What sort of traversal is this?




```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def traversal(node, out):
    if not node:
        return
    traversal(node.right, out)
    out.append(node.val)
    traversal(node.left, out)

out = []
traversal(root, out)
print(out)
```

3b) How would the code change if we were doing in-order, pre-order, and post-order traversal? What would the output be for in-order, pre-order, and post-order traversal?

3c) Binary Search Trees:

3.c.1: What does the following binary search tree look like when we insert 90?

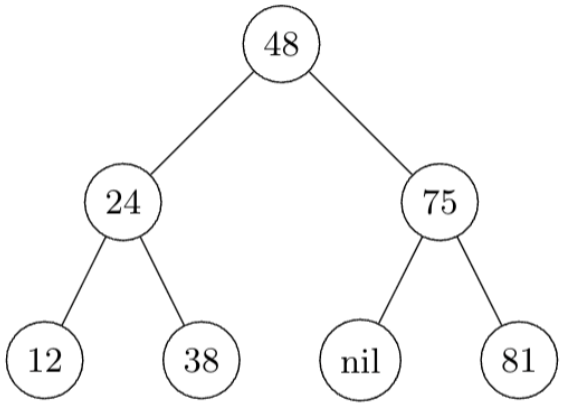


Figure: Insert 90 in the above BST!

3.c.2: What does the following binary search tree look like when we delete 400?

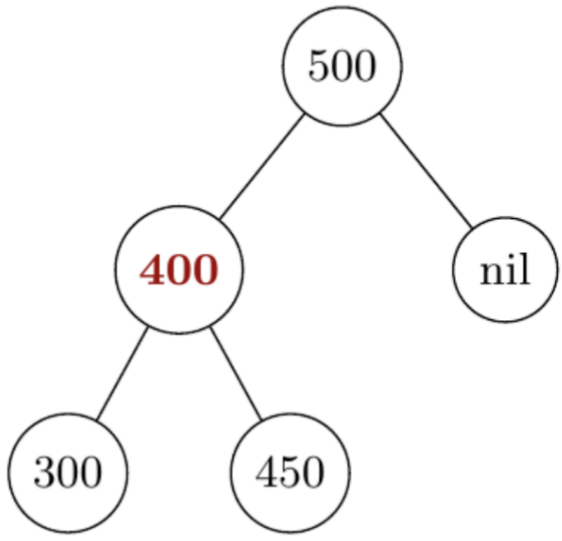


Figure: Remove 400 in the above BST!

3.c.4: What does the following binary search tree look like when we delete 600?

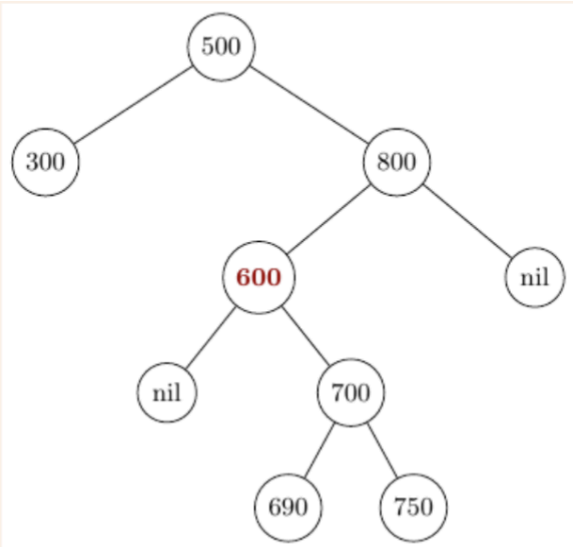


Figure: Remove 600 in the above BST!

Question 4: Code-Tracing - subset sums

```
def subset_sum(nums, target, index=0, current_subset=None, current_sum=0,
result=None):

    if current_subset is None:
        current_subset = []
    if result is None:
        result = []


    if current_sum > target:
        return result


    if current_sum == target:
        result.append(current_subset.copy())
        return result


    if index == len(nums):
        return result


    current_subset.append(nums[index])
    subset_sum(nums, target, index + 1, current_subset, current_sum +
nums[index], result)
    current_subset.pop()  # backtrack


    subset_sum(nums, target, index + 1, current_subset, current_sum,
result)

    return result
```

Example Usage

```
nums = [2, 3, 5]
target_sum = 5

solutions = subset_sum(nums, target_sum)
print("Subsets that sum to", target_sum, ":", solutions)
```

Question 5: Programming: Recursive Binary Search Tree implementation

A **Binary Search Tree (BST)** is a data structure that allows efficient insertion and search.

In a BST each node contains a value and at most two children:

1. **Left child** contains values **less than** the parent node.
2. **Right child** contains values **greater than** the parent node.

   Duplicates: you can store them in a consistent way (e.g., always on the right).

Below is a partially implemented Python class for BST. You are asked to complete the class definition for Node class' __init__() function, BST class' insert(), and search() functions for BinarySearchTree.

Tasks:

1. Node class:
   a. Store the value, left and right children are initialized correctly.
2. BST class:
   a. Implement insert() using recursion: this function returns the inserted Node object.
      i. Recurse to insert a new Node value based on BST property specified above. Capture the empty case.
   b. Implement search() using recursion: this function determines if a value exists and returns True if the element is found.
      i. Recurse to search for a Node value based on BST property specified above to determine if the Node value exists or not.

Example Usage:

```
bst = BinarySearchTree()
bst.insert(10)
bst.insert(5)
bst.insert(15)

print(bst.search(10))  # True
print(bst.search(5))   # True
print(bst.search(15))  # True
print(bst.search(20))  # False
```

```python
class Node:
    def __init__(self, value):
        """
        Initialize a BST node with appropriate information.

        Args:
            value (int): The value to store in the node.
        """
    # TODO: add appropriate information

# Binary Search Tree class
class BinarySearchTree:
    def __init__(self):

        self.root = None

    def insert(self, key):

        self.root = self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        """
        Recursive helper for insert.

        """
        # TODO: implement recursion

    def search(self, key):

        return self._search_recursive(self.root, key)

    def _search_recursive(self, node, key):
        """
        Recursive helper for search.

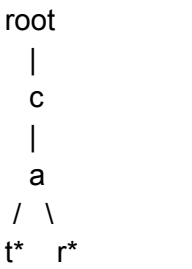
        """
        # TODO: implement recursion
```

Question 6: Programming: Recursive Tries (tree)

A Trie is a tree-like data structure used to store sequences (strings or numbers as strings).

- Each node represents a single element of the sequence (character or number).
- Each node may have multiple children, typically stored in a dictionary.
- A flag (e.g., is_end) indicates if the node represents the end of a valid key.
- Insertion and search works based on this property recursively.
- Deletion must carefully remove keys without affecting other keys sharing the same prefix recursively.

Example: insert cat and car in a tree



* indicates is_end = True.

Task

1. Implement a Node class (TrieNode):
    a. Stores a dictionary of children.
    b. Boolean flag is_end to indicate a key ends here.
2. Implement a Trie class:
    a. Contains a reference to the root node.
    b. Supports recursive insert: Adds a key to the Trie, creating nodes as needed.
    c. Supports recursive search: Checks whether a key exists in the Trie.
    d. Supports recursive delete: Check for the existence of the key. If the key exists, unmark is_end. Remove any nodes that are no longer needed (no children and is_end is False).Must preserve other keys that share the same prefix.
3. Keys can be strings or numbers (stored as strings for sequence traversal).

Note that keys can be strings or numbers (stored as strings for sequence traversal).

```
class TrieNode:
    def __init__(self):

        # TODO: add appropriate information


class Trie:
    def __init__(self):
```

```
        self.root = TrieNode()

    def insert(self, key):

        self._insert_recursive(self.root, key, 0)

    def _insert_recursive(self, node, key, index):
        """
        Recursive helper for insert.

        """
        # TODO: implement recursion

    def search(self, key):

        return self._search_recursive(self.root, key, 0)

    def _search_recursive(self, node, key, index):
        """
        Recursive helper for search.

        """
        # TODO: implement recursion

    def delete(self, key):

        self._delete_recursive(self.root, key, 0)

    def _delete_recursive(self, node, key, index):
        """
        Recursive helper for delete.

        """
        # TODO: implement recursion
```

Example Usage:

```
trie = Trie()
trie.insert("cat")
trie.insert("car")

print(trie.search("cat"))  # True
print(trie.search("cap"))  # False
```

```
print(trie.search("dog"))  # True
print(trie.search("do"))   # False
```

Question 7: Code-Tracing—Recursion. What is the output?

```
def mystery(n, a, b, c):
    if n == 0:
        return

    mystery(n - 1, b, c, a)
    print(a, "->", c)
    mystery(n - 1, c, a, b)


mystery(3, "A", "B", "C")
```

Question 8: Code-Tracing: Add Two Numbers (Linked List Representation)

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None


def add_lists(l1, l2, carry=0):

    if not l1 and not l2 and carry == 0:
        return None


    val1 = l1.val if l1 else 0
    val2 = l2.val if l2 else 0


    total = val1 + val2 + carry
    new_val = total % 10
    new_carry = total // 10


    node = Node(new_val)


    next1 = l1.next if l1 else None
    next2 = l2.next if l2 else None


    node.next = add_lists(next1, next2, new_carry)

    return node


def print_list(head):

    while head:
        print(head.val, end=" -> ")
        head = head.next
    print("None")
```

Example Usage:

```
# List 1: 2 -> 4 -> 3 (represents 342)
a1, a2, a3 = Node(2), Node(4), Node(3)
a1.next, a2.next = a2, a3

# List 2: 5 -> 6 -> 4 (represents 465)
b1, b2, b3 = Node(5), Node(6), Node(4)
b1.next, b2.next = b2, b3

# Add them
result = add_lists(a1, b1)
print_list(result)
```

Question 9: Code-Tracing - double ended queue: Round-Robin Processing

```
from collections import deque

def round_robin(tasks, quantum):

    queue = deque()
    elapsed = 0
    index = 0
    completed = []

    while index < len(tasks) or queue:

        if not queue and index < len(tasks):
            elapsed = max(elapsed, tasks[index][1])

        while index < len(tasks) and tasks[index][1] <= elapsed:
            name, arrival, remaining = tasks[index]
            queue.append((name, remaining))
            index += 1

        name, remaining = queue.popleft()

        used = min(quantum, remaining)
        elapsed += used
        remaining -= used

        while index < len(tasks) and tasks[index][1] <= elapsed:
            name_new, arrival, remaining_new = tasks[index]
            queue.append((name_new, remaining_new))
            index += 1

        if remaining > 0:
            queue.append((name, remaining))
        else:
            completed.append((name, elapsed))

    return completed
```

```
tasks = [
    ("A", 0, 5),
    ("B", 1, 3),
    ("C", 4, 1)
]

print(round_robin(tasks, 2))
```

Question 10: Programming - Determine Whether Two Binary Trees Are Identical Using Stacks

Problem Description

Write a function `identical_trees_stack(root1, root2)` that returns `True` if two binary trees are identical and `False` otherwise.

Two binary trees are **identical** when they have:

- The same structure: children exist in the same positions.
- The same keys at corresponding positions.

Each node has the attributes `key`, `lChild`, and `rChild`. Two empty trees are identical. If only one tree is empty, they are not identical.

Do not use recursion, tuples, or modify either tree.

Example: Two trees with root `4`, left child `2`, and right child `6` are identical. If one tree instead has `2` as its right child and `6` as its left child, they are not identical.

Hint:

1. Use one stack for each tree, starting with their respective roots.
2. Retrieve corresponding entries from both stacks together.
3. Consider three possibilities: both entries are empty, only one is empty, or both contain nodes.
4. When both contain nodes, compare their keys.
5. Save corresponding child positions in the same order. Remember that missing children matter when comparing structure.
6. Continue until you find a mismatch or finish comparing all positions.

Question 11: Analysis and Conceptual Questions

11a) time complexity analysis

What is the Big-O time complexity of the below function?

```
def get_first_element(numbers):
   return numbers[0]
```

Question 11b: conceptual question (Linked List Structure)

Draw the variables, references, and objects that exist after the following code executes. Label the `prev` and `next` references in your node drawings. Identify the data structure here and briefly defend your answer.

```
class Buddy:
    def __init__(self):
        self.data = None
        self.next = None
        self.prev = None

n1 = Buddy()
n1.next = Buddy()
n1.prev = Buddy()

n1.next.next = n1.prev
n1.next.prev = n1

n1.prev.next = n1
n1.prev.prev = n1.next
```

11c) time complexity analysis

What is the Big-O time complexity of the below function?

```
def sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

11d) time complexity analysis

```python
n = 5  # example value, you can change

for i in range(0, n):
    count = 0
    x = i * n
    print("i, x:", i, x)

    for j in range(0, x):
        count += 1
        print("count:", count)
```

Question 12: Code-Tracing: Permutations

What is the output? Write out all steps of the memory trace.

```
def permutation(lst, start, end):
      if start == end:
          print(lst)
      else:
          for i in range(start, end + 1):
              lst[start], lst[i] = lst[i], lst[start]     # choose
              permutation(lst, start + 1, end)            # explore
              lst[start], lst[i] = lst[i], lst[start]     # UNCHOOSE

  permutation(['B','E','V'], 0, 2)
```